# Reach: a workflow for better code

Why I built it. How I use it. What it gives back.

I use this workflow for all my coding, on projects of every size.

Clear decisions, verified behavior, and human judgment improve the result.



One small feature takes us through the workflow.

# The documentation outgrew the context

An agent can only work with so much information at once.

01 Decisions and instructions accumulated across sessions.

02 The original project reached 1,077 decision records and 662,000 lines of docs.

03 Too much documentation to fit in the agent’s context at once.

04 Relevant decisions were easy to miss; old rules competed with current ones.



A decision on disk is not necessarily in the agent’s context.

**Keep the current rules small enough to find, load, and use.**

# What the plugin actually adds

A repeatable workflow inside Claude Code, with records and checks in the project.

01 An AI coding agent can read files, edit code, and run commands.

02 Reach gives it named skills for different kinds of work.

03 Project documents hold decisions that later sessions need.

04 PowerShell scripts run verification and enforce repository rules.

**SKILLS**

Instructions for how to work

**DOCUMENTS**

The current agreement

**SCRIPTS**

Executable checks

**Reuse the workflow without rebuilding it in every chat.**

# Three kinds of work. Three ways to judge it.

Use the command that matches the decision in front of you.

### IDEATE

/reach:ideate — agree on the behavior and write the contract.

**Your design judgment**

### BUILD

/reach:build — implement the agreed behavior and prove it.

**Automated evidence**

### MILESTONE

/reach:milestone — build the experience and bring it back to you.

**Tests + your judgment**

Missing decision? Return to ideate. Update the agreement. Resume.

**You stay involved where judgment matters.**

# Start with one repository and one feature

Install once. Adopt the project. Describe the outcome you want.

01 Install the Reach plugin in Claude Code.

02 Run /reach:adopt to agree on documents, checks, and integration.

03 Our example: click beside the fireplace and have Pip walk there.

04 First version: one entrance. Movement does not open the door.



A fictional feature; no game-development experience needed.

**Adoption connects the workflow to the project you actually have.**

# 1. Decide before the agent fills in the gaps

Use /reach:ideate to turn the request into an agreed behavior.

01 You ask: “What should happen when the door is closed?”

02 Ideate explores the tradeoff: wait, refuse, or choose another destination.

03 You decide: follow a valid route; otherwise stay put and explain why.

04 Ideate records that decision in the movement contract.

**ILLUSTRATIVE CONTRACT**

When a route exists, Pip walks to the clicked destination.

When no route exists, Pip stays in place and exposes a blocked-route state.

**A plausible implementation choice becomes an explicit product decision.**

# 2. Build against that agreement

Run /reach:build. The agent implements and verifies the behavior.

01 Build reads the contract and implements movement.

02 A named test checks that a closed door leaves Pip in place.

03 Build runs the project’s configured verification stages.

04 The report distinguishes passed, failed, and skipped checks.

**ILLUSTRATIVE EVIDENCE**

closed-door-stays-put

Given: the only entrance is closed.
When: the player clicks inside.
Then: position is unchanged and the blocked-route state is set.

**“Done” has evidence you can inspect.**

# 3. Bring missing decisions back to ideate

The door closes while Pip is walking. The rule does not cover it yet.

01 Build records an Open question beside the affected claim.

02 It continues independent work where the contract is clear.

03 You return to /reach:ideate and decide: stop before the obstacle.

04 Ideate updates the contract; build reads the answer and tests it.

**BUILD FINDS A GAP**

Open: what if the route closes mid-walk?

**YOU + IDEATE DECIDE**

Stop at the last valid position.

**BUILD RESUMES**

Read the updated rule. Implement and prove it.

**Delegate implementation while keeping product decisions visible.**

# 4. Try the result yourself

Use /reach:milestone when the experience needs a human verdict.

01 Milestone builds and tests feedback over the movement behavior.

02 It walks through a real build and hands it to you.

03 You try an open door, a closed door, and a route interrupted mid-walk.

04 You confirm the result, or report what still does not work.



Does the response make sense to the person using it?

**Passing tests and accepting the experience are separate decisions.**

# The next session starts from the record

Current decisions, evidence, and open questions stay with the work.

01 The contract says what should happen now.

02 Evidence links show what was checked; missing work stays visible.

03 Open questions show where a decision is still needed.

04 Document limits force old text out. Git keeps the history.

**AGREED**

Stop before a newly blocked route.

**CHECKED**

Tests for the starting and mid-walk cases.

**NEXT SESSION**

Read the current record and pick up the work.

**Less time reconstructing the project from a long chat.**

# What I get back from the workflow

Higher-quality results are why I use it for all my coding.

01 Better design conversations: ideate creates room to question the approach.

02 Useful delegation: build keeps moving and surfaces decisions it cannot make.

03 Clearer review: contracts and evidence give me something concrete to inspect.

04 Easier restarts: the next session reads the current record.

**My experience: better results across projects of every size.**

# Why I use Reach for all my coding

Project size changes. Clear decisions and verification still matter.

01 I use the workflow on small tasks and large projects because the results are better.

02 Decide what should happen, implement it, verify it, and judge the result.

03 On team projects, the workflow works alongside the existing review process.

04 Ask: what was decided, what was checked, and what still needs me?

**The reason I keep using it: higher-quality results.**

# Appendix: install and start

Commands from this repository; run adoption in the target project.

01 Requires Claude Code, Git, and Windows PowerShell 5.1 or pwsh.

02 Terminal: claude plugin marketplace add SpoiledInk13/Reach

03 Terminal: claude plugin install reach@reach

04 Inside Claude Code: /reach:adopt

# Appendix: what the checks can prove

Gates check repository rules. Tests check behavior. People judge the experience.

01 pwsh Scripts/reach.ps1 gate — check document rules and evidence links.

02 pwsh Scripts/reach.ps1 all — run the gate and verification stages.

03 pwsh Scripts/reach.ps1 prove — exercise Reach’s supplied controls.

04 Break a guarded condition, observe the intended failure, restore, and rerun.

**A useful check can detect the mistake it is meant to catch.**

# Appendix: isolation and unattended work

Add these mechanisms when the project needs them.

01 A lane is a Git worktree: its own files, branch, staging area, and build cache.

02 Landing with -Verified refuses a merged tree different from the one checked.

03 /reach:iterate carries work across fresh agent runs and follows progress.

04 Unattended mode requires explicit opt-in and disables agent permission prompts.

# Appendix: using Reach on a team

Choose the scope that fits the team’s existing workflow.

01 Shared verification: use the gate and verification contract with normal review.

02 Personal overlay: keep your Reach documents in a locally excluded folder.

03 The owner promotes a verified task commit for team review.

04 Code, tests, and evidence marks reach the team. Reviewers still merge.

**Personal structure can coexist with the team’s review process.**